Road-Corridor Susceptibility — GIS Decision Layer

Generated 2026-09-02. Operationalizes the already-trained, already-validated Random Forest susceptibility model (`docs/model_training_report.md`) into predictions for real Sikkim road corridors. **No retraining, no new model features, no dashboard/alert-engine changes.**

STEP 12 — Critical honesty statement (read this first)

> This is a zone-level susceptibility assessment derived from a spatially validated model. It does not predict the exact time of a landslide.

- Pilot region: **Sikkim**
- Operational geography: **road-corridor**, not full-state — the training inventory (774 positive GSI records) has strong road-survey bias (96.1% within 100m of a mapped road, measured), so this layer only scores road corridors, not arbitrary terrain
- **No real-time rainfall** is included in this static susceptibility layer — rainfall belongs to the separate dynamic alert layer (`app/services/alert_engine.py`), unchanged by this work
- `susceptibility_score` is a model output probability, **not** an absolute probability of landslide occurrence, a time-to-landslide estimate, or a guaranteed risk level

Step 1 — What was inspected before writing any code

- `data/models/susceptibility_model.joblib` — the exact fitted `Pipeline` (StandardScaler + RandomForestClassifier) and `feature_cols` list from training
- `data/models/validation_report.json` — the exact risk-tier thresholds already established during training (reused here, not recomputed)
- `data/raw/sikkim_roads.geojson` — found it had **zero properties per feature** (no road class, name, or ID) because `scripts/ml/fetch_osm_roads.py` discarded OSM tags when originally built (it only needed geometry for corridor buffering at the time). Fixed and re-fetched with tags — see below.
- `app/models.py` `Zone` table — `geometry` is `POLYGON` SRID 4326, matching corridor-buffer output directly; `susceptibility_score`/`risk_tier`/`model_version` columns already match the model's output shape exactly
- `app/routers/zones.py` — confirmed `PUT /zones/{id}/susceptibility` is the only write path, and there is no `POST /zones` — zone creation has only ever gone through direct DB session (`scripts/seed_zone.py`'s pattern), reused here rather than inventing a new backend path
- No existing GIS/corridor-generation utilities existed — this is genuinely new code, built on top of the existing `scripts/ml/extract_terrain_features.py`, `scripts/ml/extract_landcover.py`, and `scripts/ml/fetch_osm_roads.py`

Step 2 — Prediction geography: real OSM roads, not invented zones

Road class filter (`MEANINGFUL_HIGHWAY_CLASSES` in `scripts/generate_zone_predictions.py`): `trunk`, `primary`, `secondary`, `tertiary`, **plus any way carrying a highway `ref` number** (e.g. `NH10`, `SH`). The second clause exists because of a real, verified OSM data-quality issue in this region: several actual National Highways — including **NH310A, "North Sikkim Highway"**, directly referenced by name in our own GSI inventory — are tagged `highway=unclassified` rather than `trunk`/`primary`. Filtering on class alone would have silently dropped genuinely major roads. Excluded: `residential`, `service`, `track`, `path`, `footway`, `steps`, and other local-access/non-vehicular classes not represented in the GSI road-corridor survey this model trained on.

Spatial extent: restricted to the pilot AOI — the same bounding box used throughout this project's data/feature/bias audits (the real extent of the 774 GSI training points). Predicting outside this box would be extrapolating beyond where the model has any evidence.

Result: **768 filtered OSM ways**, ~2009km total length.

Step 3 — Prediction units: real road segments, buffered corridors

- **Segmentation**: each road way chopped into ~500m chunks (`substring`, shapely — existing dependency). A trailing chunk shorter than 20% of the target length is merged into the previous one rather than kept as a near-zero-length sliver.
- **Corridor buffer width: 500m**, reused directly from `NegativeSamplingConfig.corridor_buffer_m` — the exact same, already-justified constant from the training pipeline's negative-sampling design (chosen because it captures 99.4% of positives' measured distance to the nearest road). Not a new number picked to look good — the same value the model's own training domain is defined by, so predictions stay within that domain.
- **Segment length** (500m): no prior project precedent for this specific choice, so it's a new, explicitly documented decision — set equal to the corridor buffer width so each unit spans roughly one corridor-width along the road.
- Stable ID per unit: `{osm_way_id}_{part_index}_{segment_index}` — deterministic, reproducible on re-run.
- **`distance_to_road` was not used anywhere in feature generation** — the corridor is only the geographic unit being scored, per instruction.

Step 4-5 — Features: identical schema to training, zonal-aggregated

Every corridor gets exactly the model's 5 source features (elevation, slope, curvature, distance_to_drainage, land_cover_class), computed with the **same extraction code as training** (`scripts/ml/extract_terrain_features.py`, `scripts/ml/extract_landcover.py` — reused directly, not reimplemented), aggregated per corridor polygon instead of at a single point:

- **Continuous features (elevation, slope, curvature, distance_to_drainage): median** of raster cells intersecting the corridor. Documented choice — robust to outlier pixels at a corridor's edge, standard for aggregating skewed terrain distributions.
- **land_cover_class: dominant (most frequent) class** among intersecting WorldCover pixels. Documented choice — the corridor's "representative" cover, matching how a person would describe it.
- Implemented with `rasterio.features.geometry_mask` on a **windowed** read (critical for the 36000×36000 WorldCover raster — masking the full array per corridor, over ~4000 corridors, would be both extremely slow and memory-heavy; verified during development, when the raster was instead reopened per corridor, memory use reached ~9GB — fixed by opening it once outside the loop).

Schema verified programmatically, not eyeballed: `verify_feature_schema()` builds the one-hot land-cover columns the identical way training did, then asserts the resulting column set exactly matches `bundle["feature_cols"]` from the saved model — raises `AssertionError` on any mismatch. Unit-tested (`tests/test_generate_zone_predictions.py`).

Step 6-7 — Prediction and risk tiers (reused, not recomputed)

- Loads the **existing** fitted pipeline via `scripts/ml/predict_zone_susceptibility.load_model_bundle()` — no retraining.
- Risk tier thresholds: **reused exactly** from `data/models/validation_report.json` (`low < 0.4233 <= moderate < 0.5953 <= high`), the same tertile cutoffs established during training. Not recomputed against the new corridor population, which would silently change what "high" means between runs.
- Output field is named `susceptibility_score` — never called an absolute probability, time-to-landslide, or guaranteed risk.

Step 8 — GIS outputs

File	Format	Notes

`outputs/gis/sikkim_road_susceptibility.geojson`	GeoJSON	Primary output — written directly via `json+shapely` (the same no-GDAL pattern already used by `fetch_osm_roads.py`), not `geopandas.to_file()`
`outputs/gis/sikkim_road_susceptibility.csv`	CSV	Same fields, no geometry
`outputs/gis/sikkim_road_susceptibility.gpkg.SKIPPED.txt`	—	**GeoPackage was not generated** — see below

GeoPackage skip reason (genuine technical incompatibility, verified twice this session): `geopandas.to_file(..., driver="GPKG")` requires `pyogrio` or `fiona`, both GDAL-backed. `pyogrio` fails to import on this machine: `An Application Control policy has blocked this file` — re-confirmed at the start of *this* task, including trying to point it at rasterio's own bundled GDAL DLL directory (didn't help — it's a policy block on pyogrio's specific binary, not a missing-PATH issue). `fiona` isn't installed and would hit the same GDAL dependency. Writing a GeoPackage by hand (it's a structured SQLite schema with specific `gpkg_contents`/`gpkg_geometry_columns` tables and WKB geometry blobs) would be exactly the "custom geospatial logic" the project's coding rule says to avoid when a library gap exists — so it wasn't attempted. GeoJSON is explicitly listed as an acceptable alternative for "lightweight frontend use" and was used instead. On a machine where `pyogrio` imports cleanly, converting the GeoJSON to GeoPackage is a single unchanged `to_file()` call — no pipeline changes needed.

Step 9 — Quality checks (all run against the real output, not a sample)

#	Check	Result
1	CRS consistency	EPSG:4326 confirmed
2	Invalid geometries	0
3	Missing feature values (post-drop)	0
4	Unexpected land-cover codes	none
5	Score range	within [0,1], see execution report below
6	Score distribution	see Low/Moderate/High counts below
7	Number of prediction units	see execution report below
8	Duplicate segment IDs	0
9	Spatial coverage	100% of filtered road segments scored
10	Suspiciously constant scores	no — real variation present (std well above the 0.01 flag threshold)

Sanity check against known inventory — NOT independent validation

The 774 positive / 774 negative training points were spatially joined against the generated corridors. **These exact points were used to train the model** — this is a pipeline sanity check (does the corridor layer point in the right direction?), not

a validation claim. See execution report below for the actual numbers. The expected pattern: mean corridor score at known-positive locations should be higher than at known-negative locations, though the gap is expected to be *smaller* than the point-level model's own training separation, because corridor-level median aggregation smooths a landslide's exact local signal across a wider 500m×1000m area.

Step 10 — Backend integration (existing contract, unmodified)

`scripts/integrate_zone_predictions.py`: 1. For each corridor, creates a `Zone` row if one with that name doesn't already exist (direct DB session — the same pattern `scripts/seed_zone.py` already established; there's no `POST /zones` endpoint to reuse, and none was added) 2. Sends `susceptibility_score`, `risk_tier`, `model_version` via a real HTTP `PUT /zones/{id}/susceptibility` request against the **unmodified, existing** endpoint 3. Verifies via `GET /zones/{id}`

No backend code was changed.

A real bug was caught and fixed during this step: the first version of `zone_name_for()` used a road's `ref` tag alone (e.g. `"NH10"`) as the match key for "does this zone already exist." Since one highway ref covers dozens of distinct 500m segments, this silently collided them — verified directly: running the full 3921-corridor integration produced only **1899** distinct zones in the database, each repeatedly overwritten by whichever same-named segment was processed last. Fixed by embedding `segment_id` into the name (`"NH10 (44848664_00_000)"`) for guaranteed uniqueness while keeping the road identifier readable. The 1898 collided rows were deleted (direct DB session, keeping only the pre-existing placeholder test zone) and the integration re-run. Verified after the fix: **3922 total zones** (3921 corridors + 1 unrelated pre-existing test zone), all names unique, tier distribution in the database matching the source file almost exactly (329 high in DB vs 328 in the file, the +1 being the test zone's own unrelated `risk_tier="high"`).

Step 11 — Frontend data contract

Endpoint/data source: `outputs/gis/sikkim_road_susceptibility.geojson` (static file, regenerate via `python -m scripts.generate_zone_predictions`) for map rendering; `GET /zones` on the live backend for the same data once integrated, per-zone.

Geometry format: GeoJSON `Polygon`, EPSG:4326 (WGS84 lat/lon) — one polygon per road-corridor segment (a buffered rectangle-ish shape around a ~500m road chunk, not the road centerline itself).

Required fields (GeoJSON `properties`, also CSV columns): | Field | Type | Meaning | |---|---|---| | `segment_id` | string | Stable unique ID, e.g. `44848664_00_000` | | `osm_id`, `highway`, `name`, `ref` | mixed | Source OSM identifiers — `ref` (e.g. `"NH10"`) is usually the most useful human label; `name`/`highway` are often null for numbered highways in this region | | `elevation`, `slope`, `curvature`, `distance_to_drainage` | float | The exact feature values used for this prediction (median-aggregated) | | `land_cover_class` | string | Dominant land-cover class in the corridor | | `susceptibility_score` | float, 0-1 | Model output — **relative** risk ranking, not an absolute event probability | | `risk_tier` | `"low"` | `"moderate"` | `"high"` | Fixed thresholds from training (see Step 6-7) — same meaning across every corridor in this file | | `model_version` | string | `random_forest-extended-v1-20260902` |

Risk-tier values: exactly `low`, `moderate`, `high` — matches the backend's `Zone.risk_tier` CHECK constraint, no remapping needed.

Score semantics: higher = more susceptible *relative to other corridors in this dataset*, based on terrain + land cover only. Not time-aware, not rainfall-aware, not an absolute probability.

Model version: `random_forest-extended-v1-20260902` — same string on every row in this run; changes only when the model is retrained.

Limitations to carry into any frontend/dashboard copy: - Road-corridor only — no coverage away from mapped major roads - Corridor-level scores are smoothed versions of point-level signal (see sanity-check note above) - `land_cover_class`'s contribution can't be fully separated between genuine anthropogenic slope destabilization and the

source inventory's own documentation bias (see `docs/duplicate_and_bias_audit.md`) - Random Forest showed a real train/CV performance gap during validation (see `docs/model_training_report.md`) — held-out spatial ROC-AUC is 0.735, not higher

---

Execution report

Real run, 2026-09-02, full pilot AOI, no sampling/truncation.

- **Prediction units generated**: 3921 (from 768 filtered OSM road ways, pilot AOI, ~2009km total length)
- **Feature coverage**: 100% — 0 corridors dropped for missing feature values, 0 invalid geometries, 0 duplicate IDs
- **Score statistics**: range [0.123, 0.883], std dev 0.134 (real variation, not constant)
- **Risk tier counts**: Low 2225 (56.7%), Moderate 1368 (34.9%), High 328 (8.4%)
- Note: this is *not* a 33/33/33 split like the training population, and that's expected/correct — the thresholds are fixed cutoffs from the balanced 50/50 training set, applied to the real road network, which (as expected for a mostly-ordinary terrain) skews toward lower risk. A roughly-even split here would actually have been suspicious.
- **Sanity check (NOT independent validation)**: 3091 known-positive training points and 1785 known-negative training points fell inside a scored corridor. Mean corridor score at positive-point locations: **0.441**; at negative-point locations: **0.392**. Direction is correct (positive > negative), and the gap is smaller than the point-level model's own training separation (0.617 vs 0.385, from the earlier point-level demonstration) — exactly as expected, since corridor-level median aggregation smooths a landslide's precise local signal across a wider 500m×1000m area.
- **Backend integration result**: ran for the full 3921 corridors, not just a sample. First attempt revealed a real bug (see Step 10 above) -- name-based zone matching collided same-ref segments, leaving only 1899 distinct zones in the database instead of 3921. Fixed (`segment_id` embedded in the zone name), bad rows deleted, re-run. Final verified state: **3922 zones in the live database** (3921 corridors + 1 pre-existing unrelated test zone), all names unique, all 3921 `PUT` requests returned `200` (22.3s total), `GET /zones` confirms the full set with the correct tier distribution.
- **Outputs**: `outputs/gis/sikkim_road_susceptibility.geojson` (3921 features), matching `.csv`, and a `.gpkg.SKIPPED.txt` explaining the GeoPackage gap